Prakhar Shekhar Parthasarthi

Full-Stack Software Engineer / Java & Spring Boot Backend / React Front-End / AWS

prakhar.mnnit.2022@gmail.com · Shahjahanpur, UP · open to relocate ·
github.com/praxstack · linkedin.com/in/prakharshekhar · prax-portfolio-one.vercel.app

47% P90 LATENCY CUT · BOOKING SURFACES

+33% CONVERSION LIFT

195k rps GET · JAVA REDIS FROM SCRATCH

20→5 ON-CALL TICKETS / SHIFT

SUMMARY

Full-stack software engineer with **3+ years at Amazon** across Payments and Travel — **owned 4 microservices end-to-end** through design, build, deploy, and on-call. Java-heavy backend (**Java 17 / Spring Boot**, JVM concurrency with `CompletableFuture`, bounded `ExecutorService`, `ConcurrentHashMap`), idempotent event-driven architecture over **SQS + Step Functions**, and typed infrastructure in **AWS CDK (Java)** — paired with **React (SSR & CSR) + Redux** for customer-facing flows. Customer-facing impact: `47% p90` reduction on booking surfaces, `33% conversion` lift, `43 bps CPT` reduction via idempotent schedule-change automation, on-call tickets cut `20+ to ~5` per shift.

CORE TECHNICAL STACK

JAVA & JVM

Java 17, Spring Boot, Spring MVC, Spring DI, Google Guice, JUnit 5, Mockito, SLF4J, Lombok, Maven, Gradle · `ExecutorService`, `CompletableFuture`, `ConcurrentHashMap`, thread-pool tuning, JVM profiling, GC analysis

DISTRIBUTED SYSTEMS

Microservices, REST, GraphQL, event-driven architecture, idempotency & dedup keys, state machines, retry with exponential backoff, DLQ handling, at-least-once semantics, TDD

AWS

Lambda (Java), DynamoDB, SQS, Step Functions, API Gateway, AppSync, CloudWatch, ECS, **CDK (Java)** — typed IaC for all service infrastructure

FRONT-END

React.js (SSR & CSR), Redux (1.5 yrs production) — customer-facing Amazon flows; HTML / CSS, JavaScript / TypeScript

DATA & STORAGE

DynamoDB, Redis, NoSQL modelling, SQL, caching strategies

OTHER LANGUAGES

Python, C++, VTL

TOOLING & PROCESS

Git, GitHub, CI/CD, Gradle JaCoCo coverage gates, Agile / Scrum, on-call, code review, incident response

EXPERIENCE

Software Development Engineer

Jul 2022 – Sep 2025

Amazon · Travel (Hotels & Flights) · Bengaluru

- **Service ownership — Spring Boot, Spring MVC.** Owned **4 customer-facing microservices** powering the hotel booking platform — search, listing, room selection, order review — across design, code review, deploy, and on-call. Built REST controllers, service-layer logic, and `@Repository` DynamoDB DAOs wired through Spring DI. Shipped Prime-exclusive discount flow and partner-discount integration with tuned cache logic — `~30 bps` drop in funnel churn.
- **JVM concurrency — CompletableFuture, bounded executors.** Refactored sequential downstream calls into `CompletableFuture.allOf()` on a bounded `ExecutorService`; parallelised 6 partner-API lookups — hotel landing p90 cut `47%` (`1.1s → 0.59s`), conversion up `+33%`. GZIP on 150–200 MB catalog payloads cut wire size `95%`, eliminating network-timeout failures.
- **Event-driven — SQS + Step Functions.** Designed idempotent Java webhook for real-time flight schedule changes (cancellations, date-changes, non-operational flights) — SQS ingestion, DynamoDB-backed dedup keys, audit logging, automated offline modifications — addressing the highest-impact customer-pain category (`43 bps CPT`). Orchestrated end-to-end booking state machine (confirm / cancel / retry) on Step Functions with Java Lambda handlers.
- **Infrastructure as code — AWS CDK (Java).** Authored all service IaC in **Java CDK** — Lambdas, DynamoDB, SQS, Step Functions, API Gateway, CloudWatch alarms. Migrated 2 services from hand-rolled CloudFormation to typed CDK stacks — compile-time infrastructure checks, `halved` rollback incidents.
- **Testing — JUnit 5, Mockito, Spring Boot.** Held `80%+` line coverage; Mockito argument captors for downstream contract verification; `@SpringBootTest` integration tests against in-memory DynamoDB. Introduced Gradle JaCoCo coverage gates in CI.
- **Operational excellence & high-bar automation.** Dove deep into multi-year on-call corpus — tickets cut from `20+ to ~5 per shift` (`~2 SDE-weeks / week` reclaimed). Java email-automation service deprecated stale WebLab experiments — config-hygiene incidents from 5–6 to `0` per on-call. Resolved `150+` production incidents; authored COEs and drove action items to closure. Built auto-insurance purchase flow (Java + React) with milestone persistence and auto-fill — `60%+` lift in renewals.

Software Development Engineer, Intern

May 2021 – Jul 2021 · Feb 2022 – Jul 2022

Amazon · Bengaluru

- Migrated legacy GraphQL resolvers to **Java** — rebuilt resolver classes, wired them to DynamoDB via Spring-injected DAOs, exposed through API Gateway + Lambda.
- Shipped **One-Click Renewal** for auto-insurance using past-purchase data — async Java pre-fill workflow that materially simplified the funnel.
- Applied OO design and **Google Guice DI** throughout; landed a clean, testable codebase with `90%+` unit-test coverage (JUnit 5 + Mockito).

SELECTED PROJECTS

Redis Server in Java

github.com/praxstack/redis-server-java

`Java 17` `Maven` `JUnit 5` `Mockito` `ConcurrentHashMap` `ScheduledExecutorService`

RESP2-compatible Redis server built from scratch. Thread-per-client on a bounded `ExecutorService`; lock-free reads via `ConcurrentHashMap`; hybrid TTL eviction (lazy check on `GET` plus active `ScheduledExecutor` sweep); graceful shutdown via `Runtime.addShutdownHook`; integration tests over real TCP sockets.

Benchmarks (redis-benchmark, 100 concurrent clients): SET `154k rps` · GET `195k rps` · INCR `128k rps` · p50 < `0.3 ms`. **42 tests, 100% pass.**

Markdown Viewer App

github.com/praxstack/markdown-viewer-app

`Vite` `Vanilla JS` `Marked.js` `PrismJS` `Mermaid` `Vitest`

Production-ready markdown viewer with real-time preview, syntax highlighting, Mermaid diagram rendering, 10 professional themes, and PDF / HTML export. Architected with modular vanilla JavaScript — zero framework overhead, sub-100 KB bundle.

Metrics: `155+` Vitest tests · real-time preview · Mermaid diagrams · PDF / HTML export.

Warp BYOK — Bedrock Proxy

github.com/praxstack/warp-byok-proxy

`Rust` `Tokio` `AWS Bedrock` `SigV4` `HTTP Proxy`

A local single-binary proxy in **Rust** that routes Warp Terminal's AI calls to a personal AWS Bedrock account (bring-your-own-key), re-signing requests with SigV4 / bearer-token auth — a low-overhead on-device gateway with no third-party data egress.

Stack: async Rust (Tokio) · on-device · AGPL-3.0.

Open-Source Contributions

Merged into danielmiessler/Fabric (Go, 42k★ AI-tooling framework) — three production PRs:

`#2044` — added AWS Bedrock bearer-token (ABSK) authentication + guided `--setup` (the same auth Claude Code uses). `+882 / -83`

`#2052` — dynamic Bedrock region fetching from botocore (40+ regions) + AWS_PROFILE conflict fix. `+252 / -15`

`#2061` — fixed a streaming-goroutine deadlock in `Chatter.Send` + unified CLI/REST strategy handling. `+494 / -42`

Open PRs under review: bytedance/deer-flow (75k★ · model-config normalization), thedotmack/claude-mem (85k★ · plugin packaging), refactoringhq/tolaria (17k★ · editor callouts).

EDUCATION

M.C.A., Computer Science · Motilal Nehru National Institute of Technology (MNNIT), Prayagraj `GPA 9.25` 2019 — 2022

B.Sc. (Hons), Computer Science · Bhaskaracharya College of Applied Sciences, University of Delhi 2015 — 2018

AWARDS & RECOGNITION

Amazon Pay Merchants Categories Champion — 100+ hotel trust-buster bugs resolved; on-call tickets cut `20+ to ~5` per shift. 2023